- Document 객체
  - 문서(text)와 그에 연결된 정보(메타데이터)를 하나로 묶는 데이터 구조
  - RAG는 문서를 쪼개고 → 임베딩하고 → 검색한 다음 → LLM에 넘겨주는 과정을 거치는데, 문서가 문자열(text)만 있으면 부족함
  - 메타데이터 활용
    - TextSplitter 사용 시 여러 개의 Document로 나뉘어도 메타데이터는 유지됨
    - 각 문서에 대해 임베딩 처리될 때에도 메타데이터는 결과에 함께 저장됨
    - 검색된 문서들을 메타데이터 기준으로 출처 확인 및 문서 필터링 가능

▼ 10-DocumentLoader

[Ref] Docs by LangChain Document loader integrations - Docs by LangChain

- PDF Loaders

PDFs

The below document loaders allow you to load PDF documents.

Document Loader	Description	Package/API
PyPDF	Uses `pypdf` to load and parse PDFs	Package
Unstructured	Uses Unstructured's open source library to load PDFs	Package
Amazon Textract	Uses AWS API to load PDFs	API
MathPix	Uses MathPix to load PDFs	Package
PDFPlumber	Load PDF files using PDFPlumber	Package
PyPDFDirectry	Load a directory with PDF files	Package
PyPDFium2	Load PDF files using PyPDFium2	Package
PyMuPDF	Load PDF files using PyMuPDF	Package
PyMuPDF4LLM	Load PDF content to Markdown using PyMuPDF4LLM	Package
PDFMiner	Load PDF files using PDFMiner	Package
Upstage Document Parse Loader	Load PDF files using UpstageDocumentParseLoader	Package
Docling	Load PDF files using Docling	Package
UnDatasIO	Load PDF files using UnDatasIO	Package
OpenDataLoader PDF	Load PDF files using OpenDataLoader PDF	Package

```
from langchain_community.document_loaders import PyPDFLoader # 로더 정의 loader = PyPDFLoader(file_path) # 문서 로딩 docs = loader.load()
```

Loader	내용	텍스트 정확도	이미지 추출	표 추출	문단 구조	속도	주요 용도
`PyPDFLoader`	가장 기본	보통	X	X	X	빠름	일반 문서
`PyMuPDFLoader`	이미지도 추출	높음	O	보통	O	빠름	이미지 + 텍스트 혼합 문서
`PDFPlumberLoader`	표 추출까지 고려	보통	X	O	보통	보통	표 추출 목적
`UnstructuredPDFLoader`	자연어 구조 단위 분할 고려	높음	O	O	O	느림	고품질 구조화 분할

- 주요 특징
  - `PyPDFLoader`
    - 평균적으로 우수 (한글 인코딩 처리, 속도, 메타데이터)
    - 페이지 단위로 데이터 로드 (e.g., 20페이지 파일 로드 시, 20개 문서로 로드)
    - 메타 데이터 : source(파일명), page(페이지번호)
  - `PDFPlumberLoader`
    - 한글 인코딩 처리 능력이 우수하고, 다양한 메타데이터 정보 제공 (Author, ModDate)
    - 문서 처리 속도가 가장 느림
  - `UnstructuredPDFLoader`
    - 가장 많은 메타데이터 제공하지만 속도가 다른 로더 대비 느림
    - 메타데이터 중 페이지번호를 제외한 다른 메타 정보가 필요하지 않다면 고려 대상에서 제외
- 텍스트 위주 : `PyMuPDFLoader` or `PyPDFLoader`
- 표가 많은 문서 : `PDFPlumberLoader`
- 자연어 구조 분할 : `UnstructuredPDFLoader`

> 위와 같이 확인 가능하나, 대상 문서에 적용하여 실험 결과 확인 후 채택 필요 👊

- JSONLoader, CSVLoader, UnstructuredHTMLLoader, TextLoader, DirectoryLoader, ...

▼ ✅ TO-DO :

- [ ] `01-PDF-Loader.ipynb`

2. Text Splitter

- 로드된 문서를 처리 가능한 작은 단위로 분할
- 처리 문서의 특징을 확인하고, 필요한 정보(메타데이터)와 처리 속도를 고려하여 선정 필요

▼ ✂️ 11-TextSplitter

[Ref] Docs by LangChain Text splitter integrations - Docs by LangChain

A. Rule-based

- CharacterTextSplitter
  - 텍스트를 특정 크기의 "청크(Chunk)" 단위로 나눔
    - 언어 모델은 입력 길이에 제한 있어 적절한 크기로 나눠야 함
    - (e.g.) GPT-3.5-turbo-0613 : 최대 토큰 길이가 4096 토큰으로 해당값 이하로 분할 필요
  - 문맥 보존
    - 단순히 텍스트 길이 기준으로 자르지 않음
    - 의미 단위 (문장 또는 단어)를 최대한 유지하면서 분할
    - 그러나 소제목은 잘 처리하지 못함 : Overlap을 적용해도 해결되지 않는 경우 많음
- TokenTextSplitter
  - 텍스트를 "토큰" 단위로 분할 → 한글 처리 시 아쉬움 많음
  - 모델이 이해하는 토큰 개수에 맞게 분할 (e.g., GPT-3.5-turbo : 4096 토큰)

B. Recursive Rule-based

- ⭐ RecursiveCharacterTextSplitter ⭐

```
from langchain_text_splitters import RecursiveCharacterTextSplitter # 분할기 정의 text_splitter =
RecursiveCharacterTextSplitter( chunk_size=250, chunk_overlap=50, length_function=len, # default: 문자수 기준 (공백 포함)
is_separator_regex=False, # 문장/문단 분리용 구분자를 일반문자열 처리 여부 ("\n\n", "\n", ".", " ") ) # 청크 분할 texts =
text_splitter.create_documents([file])
```

  - 긴 텍스트를 모델 입력에 적합한 "청크" 단위로 나누되 문맥을 최대한 보존하도록 함
  - Recursive : 여러 구분자 (eg, 문단 → 문장 → 숙어) 순서로 점점 작게 나누는 방식
  - 분할된 텍스트가 의미를 잃지 않도록 최소한 조정하여 처리
  - 다양한 구분자를 활용해 점진적으로 나눔
- RecursiveJsonSplitter
  - JSON 데이터를 처리하기 위한 텍스트 분할 도구
  - JSON의 계층적 구조(KEY-Value)를 유지하면서 분할

C. Semantic-aware

- SemanticChunker

```
from langchain_experimental.text_splitter import SemanticChunker # 분할기 정의 text_splitter = SemanticChunker(embedding) # 텍스
트 분할 texts = text_splitter.split_text(file)
```

  - 텍스트를 "의미(semantic) 유사성" 기반하여 청크를 생성
  - 청크가 문장, 문단 또는 의미적으로 연결된 문장 묶음으로 분할
  - 긴 문서를 분할하더라도 문서의 원래 흐름과 구조를 최대한 유지
  - 다른 토크나이저와 달리 chunk_size, chunk_overlap과 같은 파라미터를 사용하지 않음
- SentenceTransformersToeknSplitter

  [Ref] langchain_text_splitters.sentence_transformers.SentenceTrans...

  - sentence-transformer 모델에 특화된 텍스트 분할기
  - sentence-transformer
    - 문장 단위의 의미(semantic)를 잘 파악할 수 있도록 설계된 임베딩 모델
    - Transformer 모델에 특수한 구조(Siamese/Triplet 등)를 추가하여 문장 간 의미 비교,검색, 유사도 분석에 특화된 모델
    - 입력 토큰 제한 (eg : 384, 512 등)

D. Open Source

- HuggingFace Tokenizer ⭐

  [Ref] tokenizers

  - 속도와 성능 측면에서 개선된 버전이 지속적으로 업데이트 되고 있음
  - 신조어, 도메인 용어 등의 대응 위해 `tokenizer.trainer` 로 Finetuning 또는 단순 추가
- GPT Tokenizer

  [Ref] 🤗 transformers.models.gpt2.tokenization_gpt2_fast — transforme...

  - GPT-2 모델에서 사용하는 tokenizer로, 문장을 모델이 처리할 수 있도록 토큰 단위로 변환
  - GPT 계열에 최적화. 여타 LLM과는 호환성이 떨어짐

모델	전용 토크나이저	인코딩 방식
GPT-2, GPT-3	GPT2TokenizerFast	Byte Pair Encoding (BPE)
BERT, RoBERTa	BERTTokenizerFast	WordPiece
LLaMa, Mistral	LLaMATokenizer, MistralTokenizer	SentencePiece
Cluade, Gemini	(공개 토크나이저 없음)	

▼ ✅ TO-DO :

- [ ] `01-RecursiveCharacterTextSplitter.ipynb`
- [ ] `02-SemanticChunker.ipynb`

- [참고] 복잡한 표 데이터 처리 : 👀 Parser with VLM

3. Embedding

- 문서 또는 문서의 일부를 벡터 형태로 변환하여 문서의 의미를 수치화 함
- 사용자 질의 벡터화 시에도 동일한 모델이 적용되어야 함 🙏

▼ 🔢 12-Embedding

[Ref] Docs by LangChain Embedding model integrations - Docs by LangChain

- OpenAIEmbeddings ⭐
  - Transformer 계열로 문맥 기반의 동적인 임베딩 생성
  - 지원 모델 : `text-embedding-3-small` , `text-embedding-3-large` , `text-embedding-ada-002`

MODEL	PAGES PER DOLLARS	MTEB EVAL	MAX INPUT (TOKENS)	DIMENSION	RELEASED
text-embedding-3-small	62,500	62.3%	8191	512, 1536	2024.01
text-embedding-3-large	9,615	64.6%	8191	256, 1024, 3072	2024.01
text-embedding-ada-002	12,500	61.0%	8191	1536	2022.12

    - MTEB EVAL : Massive Text Embedding Benchmark에서의 평균 정확도, 임베딩 성능
    - PAGES PER DOLLARS : 1달러로 임베딩 할 수 있는 페이지수

  ⇒ 비용 대비 효과 측면에서 모델 선택 필요

- CashBackedEmbeddings ⭐

```
from langchain.embeddings import CacheBackedEmbeddings # 캐시 지원하는 임베딩 설정 # namespace: 기본 임베딩과 저장소를 사용하여 캐시 지원 임베딩 생성 cached_embeddings = CacheBackedEmbeddings.from_bytes_store( underlying_embeddings=embeddings document_embedding_cache=store, namespace=embeddings.model, # e.g., "text-embedding-3-small" ) # VectorDB 저장 vector_stores = FAISS.from_documents(documents, cached_embeddings)
```

  - 임베딩 계산 결과를 저장하거나 임시로 캐싱하여 다시 계산할 필요가 없도록 함
  - 중복 작업 방지 및 성능 최적화 ⇒ 성능 + 비용 + 캐시관리
  - 여러 임베딩 모델을 사용하는 경우, namespace를 별도 설정하여 1개 문서에 대한 서로 다른 캐싱 관리 가능
    - namespace : 구역 또는 폴더 역할로 같은 문서라도 임베딩 모델이 다르면 서로 다른 결과가 나오므로 각각 나눠놓는 것

```
embeddings_ada = OpenAIEmbeddings(model="text-embedding-ada-002") embeddings_3 = OpenAIEmbeddings(model="text-embedding-3-small") vectorstore_ada = FAISS.from_documents(docs, embeddings_ada, namespace="ada-002") vectorstore_3 = FAISS.from_documents(docs, embeddings_3, namespace="3-small")
```

- Open source Embedding
  - 지속적으로 업데이트 되고 있어, 한글 임베딩 특징 확인 후 실험 통해 채택 여부 결정
  - Hugging Face : huggingface.co
    - 좌편 메뉴 : Retrieval > RTEB Multilingual 선택
    - Customize this Benchmark > Language - [`eng` , `kor`] 만 선택 후 리더보드 확인
  - Allganize : 🤗 huggingface allganize/RAG-Evaluation-Dataset-KO · Datasets at Hugging Fa...
    - 전체 RAG Pipeline 관점에서 도메인 별로 성능 확인

▼ ✅ TO-DO :

- [ ] `01-OpenAIEmbedding.ipynb`
- [ ] `02-CacheBackedEmbeddings.ipynb`

▼ [참고] 임베딩 차원의 장단점

- 임베딩 차원이 클 때의 장점
  - 표현력 증가 : 더 많은 차원은 더 많은 정보를 담을 수 있어, 텍스트 간의 미묘한 의미 차이도 반영할 수 있음
  - 유사도 구분력 향상 : 고차원일수록 문서 간 유사도 계산이 더 정밀하게 작동
  - 복잡한 문맥 처리 가능 : 긴 문장, 다의어, 개념적 내용도 더 풍부하게 표현 가능
- 임베딩 차원이 클 때의 단점
  - 비용 증가 : 벡터 차원이 높을수록 임베딩 API 호출 비용 증가
  - 저장 공간 증가 : 벡터 하나당 차원수 만큼 float 저장 → 수백만개 벡터면, 수십 GB 이상
  - 검색 속도 저하 : 차원이 높을수록 유사도 개산 비용 증가
  - 과적합 가능성 : 데이터셋 대비 너무 큰 차원에 반영하면 오히려 성능이 떨어질 수 있음

1

4. Vector Store

- 임베딩 벡터들을 DB에 저장

▼ 🪣 13-VectorStore

[Ref] Docs by LangChain Vector store integrations - Docs by LangChain

- FAISS
  - FAISS, Facebook AI Similarity Search
  - 기본적으로 벡터 검색 및 유사도 매칭 위한 "라이브러리"
  - "벡터 저장 + 검색" 역할도 가능하기 때문에 Vector DB 중 하나로 불리우고 있음
  - 자체 인덱스 관리
    - `IndexFlatL2` : 가장 단순한 구조 (L2 거리 비교), 정확도는 가장 높으나 전체 비교를 하여 속도 느림
    - `IndexIVFFlat` : 여러 클러스터로 분할 후 비교, 속도는 빠르지만 정확도는 다소 아쉬울 수 있음
    - `IndexHNSW` : 그래프 기반 인덱스, 속도와 정확도 측면에서 우수
    - `IndexPQ` : 압축 기반, 메모리 절약 효과는 있지만 정확도는 낮음
- Chroma
  - LLM과 자연어 처리 어플리케이션 위한 Vector DB
  - FAISS가 고성능 벡터 검색과 대규모 데이터 처리에 적합하다면,
  - Chroma는 벡터와 메타데이터를 통합 관리한다는 점이 강점

🏃 RUNTIME

5. Retriever

- 질문이 주어지면, 이와 관련된 벡터를 벡터DB에서 검색

▼ 🔍 14-Retriever

[Ref] Docs by LangChain Retrieval - Docs by LangChain

- 주요 검색기

Retriever	활용도	활용 포인트	내용
VectorStoreRetriever	기본/필수		가장 기본적인 벡터 검색 방식으로 간단/빠르게 구축 가능
ParentDocumentRetriever	⭐	문서 구조	문단 분할 시 발생하는 문맥 손실 보완 (chunk-embedding 한계 보완)
MultiQueryRetriever	⭐	검색 성능	다양한 표현의 질의 벡터 생성 → recall 상승 (검색 정확도 향상)
EnsembleRetriever	⭐⭐	검색 성능	Sparse + Dense Retriever
ContextualCompressionRetriever	필요 시	노이즈 제거	의미 기반 압축, 노이즈 제거
LongContextReorder	⭐	검색 성능	검색 후 문서 순서 재배열

- VectorStoreRetriever
  - VectorStore에서 데이터 검색시 사용하는 검색 인터페이스
  - 임베딩 벡터 기반으로 사용자 쿼리와 유사한 데이터를 검색 → `as_retriever`
- ParentDocumentRetriever
  - 문맥이 잘리거나 청크가 너무 작아 의미 파악이 어려운 경우
  - 청크 구분
    - child chunk : 검색에 좋은 크기. 임베딩이 한 가지 의미에 집중돼 검색 정밀도가 높음
    - parent doc : 생성에 적절한 크기. LLM에게 전달할 문맥
  - child chunk 단위로 검색 → 해당하는 parent doc 반환 → 청크가 아닌 상위 문서 기반으로 응답 생성

    ⇒ child chunk → retrieval → parent 연결 → Context 강화 → 응답 생성

  ▼ 샘플 코드

```
# Parent parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1000) # Child child_splitter = RecursiveCharacterTextSplitter(chunk_size=200) # VectorDB # Child Chunk들을 임베딩하여 인덱싱 (검색 대상) # → 검색된 child 청크에서 연결된 parent 문서를 불러오기 위한 기반 vectorstore = Chroma( collection_name="split_parents", embedding_function=OpenAIEmbeddings(), ) # Parent doc 저장 : 검색된 child 청크로부터 연결된 parent 문서 검색 참조 시 사용 store = InMemoryStore() # 검색기 정의 # - child_splitter: 검색용으로 사용할 작은 청크 분할기 # - parent_splitter: 문맥 보존을 위한 원본 문서 분할기 retriever = ParentDocumentRetriever( vectorstore=vectorstore, docstore=store, child_splitter=child_splitter, parent_splitter=parent_splitter, ) # 문서 연결 : 자동으로 child chunk + parent 맵핑 생성 # - child 청크는 vectorstore에 저장되고 # - parent 문서는 store에 저장되며 # - child 청크는 해당 parent 문서 ID로 연결됨 retriever.add_documents(docs)
```

  - 청크가 어느 parent에 속하는지를 나타내는 Document ID 또는 메타데이터 기반 맵핑 (LangChain은 자동 관리)
- MultiQueryRetriever
  - Naive : 1개 질문 → 1개의 쿼리 벡터 → Top K 문서 검색
  - Multi : 1개 질문 → 질문 다양화 → N개의 쿼리 벡터 → 결과 통합+중복제거

  ▼ 샘플 코드

```
# prompt : 5개의 질문을 생성하도록 정의 # Chain 정의 custom_multiquery_chain = ( {"question": RunnablePassthrough()} | prompt | llm | StrOutputParser() ) # 사용자 질문 question = "XXX 사용법에 대해 알려주세요." # 검색기 정의 multiquery_retriever = MultiQueryRetriever.from_llm( llm=custom_multiquery_chain, retriever=db.as_retriever(), )
```

  - LLM 통해 질문을 다양화 하여 더 의미 있는 결과 도출

- ⭐ EnsembleRetriever ⭐ - Hybrid Retriever
  - Sparse Retriever(키워드) + Dense Retriever(의미)
  - ▼ 샘플 코드

```
# Sparse Retriever (BM25) # 키워드 매칭에 강하고 빠름 bm25_retriever = BM25Retriever.from_texts(doc_list) # Dense Retriever (FAISS + OpenAIEmbeddings) # 의미 기반 유사도 검색에 강함 embedding = OpenAIEmbeddings() vectorstore = FAISS.from_texts(doc_list, embedding) faiss_retriever = vectorstore.as_retriever(search_kwargs={"k": 2}) # Top 2개 문서 검색 # Ensemble Retriever # BM25 (Sparse) + FAISS (Dense) 결과를 결합하여 더 풍부한 검색 결과 제공 # weights는 각 retriever의 기여도 비율을 의미 ensemble_retriever = EnsembleRetriever( retrievers=[bm25_retriever, faiss_retriever], weights=[0.5, 0.5] # 동일 비중으로 정의한 경우 )
```

  - Semantic Search가 유사 의미를 가지는 문서 검색에 유리하나,
  - 특정 키워드가 반드시 포함되어야 하는 검색 시 유사 단어가 포함된 문서가 검색되도록 처리 할 필요 있음
- LongContextReorder - Reranker

  [Ref] Lost in the Middle : arxiv.org

  - 기존 : 중요도 기준으로 내림차순 정렬
  - 변경 : 매우 중요 - 덜 중요 - 중요 순서로 정렬
  - Trade-off
    - 정확도 vs 처리 시간
    - 성능 향상 vs 계산 비용
    - 검색 속도 vs 검색 정확도
    - 시스템 요구사항 충족

▼ ✅ TO-DO :

- [ ] `01-VectorStoreRetriever.ipynb`
- [ ] `02-ParentDocumentRetriever.ipynb`
- [ ] `03-MultiQueryRetriever.ipynb`
- [ ] `04-EnsembleRetriever.ipynb`
- [ ] `06-LongContextReorder.ipynb`

- [참고] Sparse Retrieval :
  - BM25는 query - docs에 같은 단어가 있어야지만 잡아내는 exact matching 방식
  - `EnsembleRetriever`로 검색 정확도를 높일 수는 있지만, sparse 자체의 성능을 높이는 방식도 고려 해야 함
  - SPLADE

6. Prompt

- 검색된 정보를 바탕으로 언어 모델 위한 질문 구성

7. LLM

- 구성된 프롬프트를 사용하여 언어 모델이 답변 생성

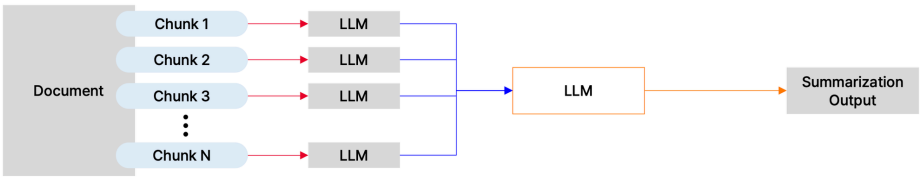
8. Chain

- 앞선 모든 과정을 하나의 파이프라인으로 묶어주는 체인 생성

▼ ⛓️ 15-RAG : Summarization Task

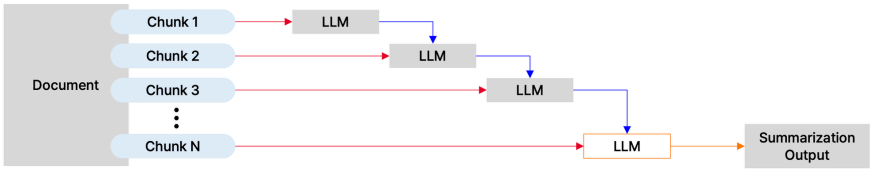
- Stuff : 전체 문서를 한번에 요약
  - 문서 목록을 가져와서 모두 프롬프트에 삽입
  - 문서가 작은 경우 효과적
- Map-Reduce : 분할 요약 후 일괄 병합
  - 긴 문서를 요약하는 대표적인 방법 중 하나
  - map : 문서를 청크 단위로 나누고
  - reduce : 각 청크의 요약을 결합

❑ Reduce : 각 청크의 요약을 병렬처리한 후에 결합하는 방식



- Map-Refine : 분할 요약 후 점진적인 병합
  - 문서 순서를 유지하면서 점진적으로 요약 개선
  - map → refine → (반복) → 최종 요약
    - refine : 생성된 요약들을 순차적으로 처리하여 최종 요약을 점진적으로 개선
    - 각 단계에서 이전 요약과 새로운 청크의 정보를 결합하여 요약 갱신 → 문서 순서 유지

❑ Refine : 생성된 요약들을 순차적으로 처리하여 최종 요약을 점진적으로 개선



- Chain-of-Density : N번 반복 실행 통해 엔티티를 보완하여 요약 개선
  - 주요 내용
    - Chain-of-Density (2023) : arxiv.org
    - 요약(Summarization)의 정보 밀도 문제에 주목
    - 기존의 요약은 핵심 정보를 생략하거나 지나치게 단순화하여 정보 손실(lossy summary)이 발생하는 경우가 많음
    - 핵심 프로세스
      - 초기 요약 : 문서에서 1~3개의 핵심 엔티티만 포함하는 간결한 요약 생성
      - 반복 요약 : 각 반복 단계에서 새로운 핵심 엔티티를 추가하여, 길이는 유지하되 정보 밀도는 증가하도록 유도
      - 총 5번 반복적으로 진행하며, 정보 밀도가 높은 고밀도 요약문 생성
  - 핵심 기여
    - Chain-of-Density 프롬프트 설계
    - 원본 문서 보다 더 핵심적인 정보가 반영된 요약문 구성 가능
    - ROUGE, Entity Recall, GPT-4 기반 평가 등을 통해 정보 풍부성, 정밀성, 충실도 검증
  - 연구적 의의
    - 요약의 새로운 관점 제시 : 기존 요약은 짧고 읽기 쉬운 것이지만, CoD는 "짧지만 정보 밀도가 높은 요약"
    - LLM 프롬프트 엔지니어링 필요 : 프롬프트 체인 통해 요약 정교화 가능성을 보여줌
    - 요약 품질 향상 뿐만 아니라, 고밀도 학습 데이터 생성 방법으로도 응용
- Clustering Map-Refine

  [Ref] GitHub langchain-tutorials/data_generation/5 Levels Of Summarizatio...

  - 문서 청르를 N개의 클러스터로 나누고, 각 클러스터의 중심점에 가까운 문서를 Refine 요약

1

▼ ✅ TO-DO :

- [ ] `04-Summary.ipynb`

💯 RAG 평가

- 성능 좋은 LLM이라 하더라도, 검색 단계에서 관련 문서를 가져오지 못하면 좋은 답변을 기대할 수 없음
- 평가 방법
  - Retrieve : 정답 문서가 포함되어 있고 & 몇 번째 위치하는지를 함께 봐야 함
    - Hit Rate@K : 상위 K개의 검색 결과 안에 정답 문서가 포함되어 있는가
    - MRR, Mean Reciprocal Rank : 정답 문서가 검색 결과의 몇 번째에 나오는가
  - Generation : LLM-as-a-Judge
- 평가 데이터셋
  - 정답 청크를 LLM에게 전달하고, 해당 내용으로 답할 수 있는 질문을 생성하도록 함
  - (질문, 정답청크ID) 쌍을 Ground Truth로 활용
- ✅ `14-Retriever/10-Retriever-Evaluation.ipynb`
- [참고] 🧑‍⚖️ LLM-as-a-Judge 편향성

🪴 단계적 접근

🫸 RAG, Retrieval Augmented Generation

Chain

file path : `.../langchain-v1/15-RAG/`

- RAG Basic
  - file : `01-RAG-Basic.ipynb`
  - 내용 : Naive RAG 단계 전개
  - ▼ Key blocks :

```
chain = ( { "context": retriever | format_docs, "question": RunnablePassthrough(), } | prompt | llm | StrOutputParser() )
```

    - `RunnablePassthrough()` : 사용자가 입력한 질문 문자열을 그대로 다음 단계로 넘겨주는 역할
    - `|` 연결 방식 : LangChain에서 RAG를 순서대로 조립하는 가장 기본적인 패턴

- RAG Advanced
  - file : `02-RAG-Advanced.ipynb`
  - 내용 : 01번의 기본 파이프라인 위에 실무에서 필요한 세 축 - 신뢰성, 검색 품질, 평가 - 를 하나씩 추가

- Key blocks :
  - 사전 준비
    1. 평가를 위한 청크 id 부여

```
for i, d in enumerate(split_docs): d.metadata["chunk_id"] = i
```

       - 검색 평가를 진행하려면 "정답이 들어있는 청크를 실제로 찾아냈는가"를 판별할 수 있어야 함
       - 이를 위해 분할 직후 각 청크에 공유id를 매겨 두는 사전 준비 단계

    2. 가드레일

```
def format_docs_with_source(docs): return "\n\n".join( f"[p.{d.metadata.get('page', '?')}] {d.page_content}" for
d in docs ) prompt = ChatPromptTemplate.from_template( """... Context에 답이 없으면 반드시 "제공된 문서에서 답을 찾을 수 없습니
다." 라고만 답하세요. 절대 추측하지 마세요. 답변 끝에 참고한 페이지를 [p.N] 형식으로 표기하세요. ..."""  )
```

       - 출처 표기 : 컨텍스트에 페이지 번호 태그를 붙여 LLM이 답변 끝에 근거 페이지를 인용하도록 함
       - 명시적인 답변 : "문서에 없으면 모른다고 답해라"는 규칙을 프롬프트에 명시하여 환각 억제
  - 신뢰성 - 답변과 근거 문서 함께 반환

```
# (인용용) 답변 + 근거 문서를 함께 반환하는 체인 answer_chain = ( RunnablePassthrough.assign(context=lambda x:
format_docs_with_source(x["context"])) | prompt | llm | StrOutputParser() ) # 실행 rag_with_sources = RunnableParallel(
context=retriever, question=RunnablePassthrough(), ).assign(answer=answer_chain)
```

    - `RunnableParallel` :
      - 검색된 원본 문서(`context`)와 최종 답변(`answer`)를 한 결과에 함께 담아
      - 답변 근거를 추적할 수 있는 구조로 만듦
  - 검색 품질
    1. Hybrid Search

```
bm25 = BM25Retriever.from_documents(split_docs) dense = vectorstore.as_retriever(search_kwargs={"k": 4})
hybrid_retriever = EnsembleRetriever(retrievers=[bm25, dense], weights=[0.5, 0.5])
```

       - Sparse : 키워드 기반 검색(BM25)
       - Dense : 임베딩 기반 검색
       - `EnsembleRetriever` : 두 검색 결과를 가중치(`weights`)로 조절하여 서로 보완하게 만듦

    2. Query Rewrite

```
rewrite_chain = rewrite_prompt | llm | StrOutputParser() search_question = rewrite_chain.invoke({"question":
raw_question})
```

       - `rewrite_chain` : 사용자의 모호한 질문을 검색 직전에 LLM이 키워드 중심으로 다시 작성하게 함
       - 같은 검색이라도 더 관련성 높은 문서를 더 많이 찾아내는 효과 기대 : recall 향상

- Agentic RAG 평가
  - file : `01-Agentic_RAG.ipynb`